Eliminating Hidden Compute-Bound Bottlenecks and Evaluating Compression Robustness for Edge Vietnamese License Plate Recognition

Huy Ho[1*], Khanh Nguyen[1], Hy Le[1], Tuan Tran[1], Khoi Pham[1], Triet Nguyen[1]

[1]Department of Computer Science and Engineering, Research Group on Embedded Computer Vision, Ho Chi Minh City, Vietnam.

*Corresponding author(s). E-mail(s): hohuytencent@gmail.com;

Abstract

Deploying Automatic License Plate Recognition (ALPR) on low-cost edge hardware such as the Raspberry Pi requires balancing memory limits, compute throughput, and accuracy under adverse field conditions. Prior edge surveillance work by Jagtap and Dhawale (2025) demonstrated YOLOv8 pruning and INT8 quantization on Raspberry Pi 4 alongside Tesseract OCR, yet left critical reproducibility gaps—undisclosed datasets, ambiguous OCR metrics, and omission of modern deep OCR compute costs. In this work, we design, optimize, and rigorously benchmark a zero-PyTorch, two-stage Vietnamese ALPR pipeline targeting both single-line car plates and two-line square motorcycle plates. First, migrating YOLOv8 to Pure ONNX Runtime with dynamic `QUInt8` quantization eliminates PyTorch runtime overhead, reducing deployment container images below 350 MB, trimming detector RAM initialization by 83.8% (330.96 MB to 53.61 MB), and shrinking model weights by 71.2%–72.6% while surviving constrained containers down to 256 MB RAM. Second, Docker cgroup stress testing reveals a shift from memory-bound to severe OCR compute-bound latency (85%–95% of total runtime), traced to RapidOCR's default `limit_side_len=736` minimum-side upscaling of small plate crops. By combining $\mathbf{480 \times 480}$ Pareto resolution scaling, core-matched ONNX intra-op threading, adaptive ROI capping ($\boldsymbol{H_{\max} = 96}$ px), `limit=320 max`, no-dilation DBNet inference, and disabled angle classification, we accelerate OCR by $\mathbf{5.85\times}$ and reduce end-to-end latency on simulated Raspberry Pi 4 from 4,184.24 ms to 668.74 ms ($\mathbf{6.26\times}$ speedup). Third, to replace non-reproducible weather claims, we establish a transparent two-tier robustness benchmark: 300 clean baseline images (Source A) expanded into 9,000 synthetic images across 10 perturbation scenarios and 3 severity

levels with verified bounding-box lineage, paired with 60 real-world challenging images (Source B). Evaluating 3,000 corrupted samples per model shows that INT8 compression preserves environmental robustness nearly identically to FP32 ($\mathbf{\Delta}$mAP@50 $= \mathbf{-0.09}$ pp, $\mathbf{\Delta}$EM $= \mathbf{-0.47}$ pp), while pinpointing early deep OCR collapse under motion blur, partial occlusion, and two-line slanted motorcycle plates as the primary frontier for future edge ALPR research.

Keywords: Edge ALPR, YOLOv8 Quantization, ONNX Runtime, Raspberry Pi, Deep OCR Over-Processing, Robustness Benchmark

1 Introduction

Automated vehicle access control at residential compounds, industrial campuses, and toll gates in Vietnam relies heavily on Automatic License Plate Recognition (ALPR). Unlike regions dominated by uniform single-row plates, Vietnamese urban traffic comprises over 70% motorcycles carrying compact two-line square plates alongside single-line and square car plates. Deploying reliable two-stage ALPR—first localizing the plate region and then reading alphanumeric sequences—on inexpensive single-board computers such as the Raspberry Pi 4, 3, or 2 remains challenging due to tight memory ceilings (512 MB–2 GB RAM) and modest ARM CPU compute budgets.

A recent study by Jagtap and Dhawale [1] (hereafter *Paper 23*) investigated YOLOv8-based helmet and number plate detection on Raspberry Pi 4 using L1 weight pruning, dynamic INT8 quantization, ONNX Runtime, and Tesseract OCR 5.0 [2]. While establishing YOLOv8 as a strong detector baseline, reproducing and extending Paper 23 for Vietnamese edge ALPR exposes five concrete technical and methodological gaps:

1. **Domain mismatch of classical OCR:** Paper 23 relies on Tesseract 5.0 over Indian single-line plates without releasing OCR code or line-ordering heuristics. Classical thresholding and Tesseract frequently scramble character order across Vietnamese two-line motorcycle plates.
2. **Hidden OCR compute bottlenecks under modern deep OCR:** Replacing Tesseract with modern lightweight deep OCR (such as RapidOCR/PP-OCR [3, 4]) resolves Vietnamese character layout accuracy, yet immediately shifts execution from memory-bound to severely compute-bound on ARM CPUs—an interaction left unmeasured when Paper 23 reports < 200 ms total latency alongside a 32 ms detector time.
3. **Unexplored input scaling and SIMD thread contention:** Fixing detector inputs at 640×640 ignores quadratic matrix scaling $\mathcal{O}(W \times H)$ and thread-oversubscription penalties across 1–4 core ARM processors.
4. **Black-box dataset opacity:** Paper 23 describes ∼1,000 collected images augmented above 5,000 samples without public access, train/validation/test split ratios, or per-condition sample sizes in its weather breakdown.

5. **Ambiguous evaluation metrics:** Paper 23 reports "OCR Accuracy (%)" across weather categories without mathematical definitions separating character accuracy from strict whole-plate Exact Match (EM) or accounting for missed detections.

To close these gaps, this paper presents an end-to-end engineering optimization and quantitative robustness benchmark for edge Vietnamese ALPR. Our five main contributions are summarized as follows:

- **Zero-PyTorch pure ONNX edge architecture:** We decouple inference from PyTorch/Ultralytics dependencies (`>1.5 GB`) inside a `<350 MB` Debian Bookworm Docker runtime, demonstrating that dynamic `QUInt8` quantization shrinks detector memory footprint by 83.8% and weight storage by 71.2%–72.6% while surviving extreme 256 MB RAM stress tests without out-of-memory kills.
- **Hardware-matched Pareto scaling and hidden over-processing elimination:** By profiling simulated Raspberry Pi 4/3/2 hardware under Docker cgroups, we identify 480×480 dynamic input resolution ($1.64\times$–$2.07\times$ detector speedup) with core-pinned ONNX threads, and uncover an implicit upscaling bug in default RapidOCR (`limit_side_len=736, limit_type='min'`) that inflated tiny cropped plates into megapixel tensors. Capping ROI height at $H_{\max} = 96$ px, enforcing `limit=320 max`, disabling morphological dilation, and bypassing angle classification yields a $5.85\times$ OCR speedup and $6.26\times$ end-to-end acceleration (668.74 ms / ~ 1.50 FPS on Pi 4).
- **Vietnamese two-line post-processing:** We formulate deterministic formatting and digit/letter disambiguation rules tailored to Vietnamese four-digit and five-digit plates across one-line and two-line layouts.
- **Open, reproducible two-tier robustness benchmark:** We release a structured benchmark pairing 300 clean baseline images (**Source A**, balanced 150 cars / 150 motorcycles) expanded into 9,000 synthetic corrupted variants across 10 physical perturbations and 3 severities with exact bounding-box and homography lineage, alongside 60 real-world adverse gate images (**Source B**) annotated with physical brightness and Laplacian sharpness tags.
- **Rigorous FP32 vs. INT8 robustness and failure analysis:** Across 64 robustness runs and 12 clean multi-resolution runs using formal all-points AP@50, end-to-end Exact Match, weighted Levenshtein CER, and ΔIoU, we establish that INT8 quantization induces virtually zero robustness loss relative to FP32 (ΔmAP@50 $= -0.09$ pp, ΔEM $= -0.47$ pp) and isolate early deep OCR failure modes under blur, occlusion, and slanted two-line motorcycle plates.

2 Related Work

2.1 Edge Object Detection and Model Compression

Single-stage detectors led by the YOLO family [5] have become the standard front-end for real-time ALPR on embedded hardware. Jagtap and Dhawale [1] benchmarked Faster R-CNN, SSD, YOLOv5, YOLOv7, and YOLOv8 on Raspberry Pi 4, reporting 94.9% mAP@0.5 and 32 ms detector latency for YOLOv8 combined with pruning and INT8 ONNX conversion. However, pruning unstructured weights alone yields sparse

tensors whose speedups rarely materialize on general-purpose CPU kernels without sparse-aware compilers, whereas dynamic `QUInt8` quantization inside ONNX Runtime directly reduces memory bandwidth and weight storage. Furthermore, static 640×640 exports prevent resolution adaptation across heterogeneous ARM tiers.

2.2 Scene Text and License Plate Recognition Engines

Classical OCR pipelines such as Tesseract [2] combine adaptive binarization and connected-component line finding, performing reasonably on flat, high-contrast Western or Indian single-row plates [1] but struggling with dense two-row motorcycle plates and uneven illumination. End-to-end CNN-CTC recognizers such as LPRNet [6] eliminate recurrent layers and run rapidly on desktop hardware, yet still require tight, de-warped plate crops and dedicated retraining when plate aspect ratios diverge. Multi-stage lightweight scene-text engines built on Differentiable Binarization (DBNet) [3] and lightweight CRNN recognizers, exemplified by PP-OCR [4] and packaged as ONNX Runtime workers via RapidOCR, accurately handle multi-line layouts out of the box. Nevertheless, their default text-detection settings assume full document or street scenes rather than small cropped plate regions, creating hidden over-processing overhead on embedded CPUs that has received little attention in edge ALPR literature.

2.3 Robustness Benchmarking Under Corruptions

In image classification, Hendrycks and Dietterich [7] formalized corruption benchmarks (e.g., blur, noise, brightness, and compression across severity tiers) to decouple clean accuracy from distributional robustness. By contrast, edge ALPR evaluations frequently rely on private captures or qualitative weather buckets [1], omitting sample counts, bounding-box transformation checks under geometric warp, and strict separation between detection misses and character errors. Table 1 contrasts Paper 23 against the methodology and empirical scope of the present study.

3 Proposed Edge ALPR Architecture and Optimizations

3.1 Two-Stage Modular Pipeline Overview

Figure 1 illustrates our modular two-stage architecture implemented in `pipeline/alpr_pipeline.py`. Given a BGR input frame, Stage 1 performs aspect-preserving letterboxing to a dynamic target square shape (S, S) with neutral gray padding $(114, 114, 114)$, converts channels to RGB normalized in $[0, 1]$, and runs YOLOv8 via `ONNXPlateDetector`. Candidate detections are filtered by confidence threshold τ_{det} and OpenCV Non-Maximum Suppression ($\tau_{\text{NMS}} = 0.45$), then rescaled back to original image coordinates to crop plate Regions of Interest (ROIs). Stage 2 passes each cropped ROI with confidence $\geq \tau_{\text{ocr}}$ through our de-bloated `RapidOCRRecognizer` (or optional `LPRNetRecognizer`), followed by Vietnamese license plate regex normalization. Unlike the interactive demo mode, benchmark

Table 1 Methodological and experimental comparison against baseline Paper 23 (Jagtap & Dhawale, 2025).

Criterion	Paper 23 [1]	This Study
Inference stack	PyTorch / ONNX hybrid on Raspbian	Pure ONNX Runtime, Zero-PyTorch (<350 MB Debian Docker)
Memory footprint	Not detailed	53.61 MB init RAM (−83.8% vs. 330.96 MB PyTorch), 256 MB-safe
Weight compression	Reported ∼ 40% size reduction	−72.6% static (11.70 → 3.20 MB); −71.2% dynamic (11.94 → 3.44 MB)
OCR subsystem	Tesseract 5.0 (unspecified config, 1-line plates)	Tuned RapidOCR (DBNet+CRNN) & LPRNet CTC + VN 1L/2L Regex
OCR optimization	None reported	ROI $H_{\max} = 96$, `limit=320 max`, no dilation, `use_cls=False` (5.85× faster)
Input resolution	Fixed 640 × 640	Dynamic ONNX; Pareto 480 × 480 (1.64×–2.07×) + thread pinning
End-to-end latency	Reported < 200 ms (OCR cost omitted)	Transparent full-pipeline: 668.74 ms (∼ 1.50 FPS) on Pi 4 (6.26× gain)
Dataset transparency	Black-box (∼ 1,000 raw, undisclosed splits)	Open 2-tier: Source A (300 + 9,000 corrupted) & Source B (60 field images)
Metrics rigor	Undefined "OCR Accuracy (%)"	Explicit all-points mAP@50, EM (1L/2L), Levenshtein CER, Deg%, ΔIoU

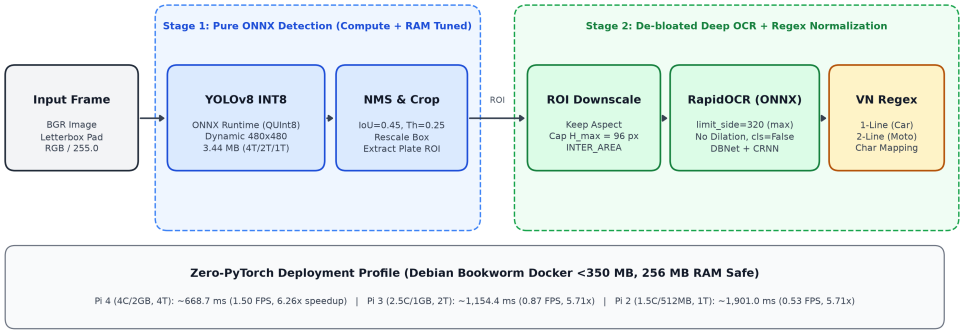


Fig. 1 Optimized zero-PyTorch two-stage edge ALPR pipeline, integrating dynamic 480 × 480 YOLOv8 INT8 detection, hardware-matched intra-op threading, adaptive $H_{\max} = 96$ px ROI downscaling, de-bloated RapidOCR DBNet/CRNN inference, and Vietnamese 1-line/2-line regex normalization.

execution strictly disables full-frame fallback OCR when zero boxes are detected (`fallback_ocr=False`) so that detection misses honestly penalize end-to-end recognition scores.

3.2 Zero-PyTorch ONNX Conversion, Pruning, and INT8 Quantization

Standard Ultralytics PyTorch inference requires installing `torch` and `ultralytics` (> 1.5 GB disk footprint) and allocates 330.96 MB resident RAM upon model initialization, causing immediate Linux OOM termination on 256 MB–512 MB edge boards. In `pipeline/optimize.py`, we convert the YOLOv8n detector (`best.pt`, 5.97 MB)

to simplified ONNX graph format (opset 18 via `onnxslim`) and apply ONNX Runtime dynamic weight-only `QUInt8` quantization (`quantize_dynamic`). An initial static 640×640 ONNX graph (11.70 MB) compressed to 3.20 MB (-72.6%), while the subsequent dynamic-input ONNX export supporting arbitrary (H, W) (11.94 MB) compressed to 3.44 MB (-71.2%). We also implement 25% L1-norm unstructured pruning across all `Conv2d` layers in PyTorch, attaining 24.91% weight sparsity (750,112 zeroed parameters in `best_pruned.pt`, 11.78 MB); however, because dense ONNX CPU kernels do not accelerate unstructured sparsity natively without sparse runtimes, production edge deployment uses pure INT8 ONNX weights (`best_int8.onnx`) executed strictly with `onnxruntime`, `opencv-python-headless`, `numpy`, `pillow`, `psutil`, and `rapidocr-onnxruntime`.

3.3 Dynamic Resolution Scaling and Thread Pinning

Since convolutional compute scales quadratically with spatial dimensions S^2, we export YOLOv8 with dynamic spatial axes and evaluate scaling $S \in \{640, 512, 480, 384, 320\}$. Letterbox coordinate inversion transforms predicted box $[x_1, y_1, x_2, y_2]$ from network space back to source pixels (W_0, H_0) via scale ratio $r = \min(S/H_0, S/W_0)$ and half-padding offsets (d_w, d_h):

$$x_{\text{orig}} = \text{clip}\left(\text{round}\left(\frac{x_{\text{net}} - d_w}{r}\right), 0, W_0\right), \quad y_{\text{orig}} = \text{clip}\left(\text{round}\left(\frac{y_{\text{net}} - d_h}{r}\right), 0, H_0\right). \tag{1}$$

Simultaneously, in `ONNXPlateDetector`, we set `inter_op_num_threads = 1`, enable full graph optimization (`ORT_ENABLE_ALL`), and pin `intra_op_num_threads` directly to physical core availability (4 threads for Pi 4, 2 threads for Pi 3, 1 thread for Pi 2/Zero) to eliminate context-switching overhead on weak ARM cores.

3.4 Eliminating Hidden Deep OCR Over-Processing

Docker resource-limit stress tests (Section 5.1) uncovered that even after cutting detector latency below $\sim$280 ms on Pi 4, total pipeline latency remained 5.34–34.40 s, with RapidOCR accounting for 85%–95% of runtime. Inspecting `rapidocr_onnxruntime` internals revealed a severe implicit bottleneck: its default DBNet text detector enforces `det_limit_side_len = 736` with `det_limit_type = 'min'`. Consequently, every compact cropped license plate ROI (typically 90×45 px to 180×80 px) was silently upscaled $5\times$–$10\times$ so its shorter edge reached 736 px, forcing DBNet to process multi-megapixel feature maps on a Raspberry Pi CPU.

We eliminate this redundant computation in `pipeline/recognizer_rapid.py` through four synergistic changes that require zero model retraining:

1. **Disable angle classification (`use_cls=False`):** Security-gate cameras maintain upright orientation, skipping 180° text-angle classification saves 0.87–2.28 s per frame across Pi tiers.
2. **Invert DBNet resize constraint (`det_limit_side_len=320`, `det_limit_type='max'`):** Caps the longest edge at 320 px rather than stretching the short edge to 736 px, suppressing implicit multi-megapixel upscaling.

3. **Remove morphological dilation (`det_donot_use_dilation=True`):** Avoids extra binary post-filtering passes over DBNet probability maps.
4. **Adaptive ROI downscaling ($H_{\max} = 96$ px):** Whenever cropped ROI height $h > 96$ px, we downsample prior to OCR via area interpolation:

$$(w', h') = \left(\max\left(16, \left\lfloor w \cdot \frac{96}{h} \right\rfloor\right), 96 \right). \tag{2}$$

3.5 Vietnamese Dual-Line Post-Processing

Vietnamese registration plates follow structured prefixes comprising a two-digit province code (11–99), registration series characters, and four- or five-digit registration numbers arranged either on a single row (cars) or stacked across two rows (motorcycles and square car plates). In `pipeline/postprocess.py`, raw recognized text fragments are stripped of non-alphanumeric symbols and uppercased into sequence c. Concatenated tokens matching 5-digit pattern `^([0-9]{2}[A-Z0-9]{1,3})([0-9]{3})([0-9]{2})$` or 4-digit pattern `^([0-9]{2}[A-Z0-9]{1,3})([0-9]{4})$` are normalized into canonical `SS-NNN.NN` and `SS-NNNN` forms. When RapidOCR returns ≥ 2 separate line segments (l_1, l_2), l_1 is treated as the province/series header and l_2 passes through visual character-to-digit mapping (`O/D/Q`→`0`, `I/L`→`1`, `Z`→`2`, `E`→`3`, `A`→`4`, `S`→`5`, `G`→`6`, `T`→`7`, `B/R`→`8`) before formatting the lower numeric group.

4 Experimental Setup and Reproducible Benchmark Protocol

4.1 Hardware Simulation Profiles and Host Environment

Phase 1–5 hardware optimization experiments were conducted inside lightweight Debian Bookworm Docker containers (`Dockerfile.rpi4`, image size < 350 MB) using strict Linux cgroup CPU and memory quotas to emulate three embedded tiers:

- **Raspberry Pi 4 profile:** 4.0 CPU cores, 2048 MB RAM (`threads=4`).
- **Raspberry Pi 3 profile:** 2.5 CPU cores (2.0 cores in stress test), 1024 MB RAM (`threads=2`).
- **Raspberry Pi 2 / Zero 2W profile:** 1.5 CPU cores (1.0 core in stress test), 512 MB RAM, plus an extreme 256 MB RAM survival test (`threads=1`).

Note that cgroup-throttled `x86_64` Docker execution isolates CPU core and RAM budgets accurately, but reflects relative scaling ratios rather than exact ARM Cortex-A72/A53 clock-for-clock instructions.

Full Phase 6 batch evaluation (`evaluation/run_benchmark.py`) ran on a Windows 11 AMD64 host (`Python 3.14.5`, `onnxruntime 1.30.0`, `opencv-python-headless 4.13.0.92`, `rapidocr-onnxruntime 1.2.3`, `threads=4`) using unified detector confidence $\tau_{\text{det}} = 0.001$ (retaining low-confidence proposals for exact all-points AP@50 integration), OCR confidence gating $\tau_{\text{ocr}} = 0.25$, and $\tau_{\text{NMS}} = 0.45$. Process memory (MiB) was sampled at 10 ms intervals via `psutil`.

Cross-platform reproducibility was further verified inside `Dockerfile.benchmark` (Debian Linux headless with `libgl1`, `libglib2.0-0`, `libgomp1`, and explicit removal of GUI `opencv-python` to prevent `libxcb.so.1` import failures).

4.2 Two-Tier Dataset Strategy and Synthetic Perturbation Suite

To prevent both simulated overfitting and opaque dataset reporting, we construct a two-tier evaluation suite (`0` overlapping filenames between sources; `0 UNKNOWN` labels):

1. **Source A (Clean Controlled Baseline, $N = 300$):** Clean 640×640 images balanced evenly between cars (150 images) and motorcycles (150 images), corresponding to 135 one-line plates and 165 two-line plates. Clean baseline labels combine verified COCO-format bounding boxes [8] with exploratory OCR annotations (`labels_verified=false`).
2. **Source A Synthetic Robustness Cohort ($N = 100$ subset $\times 30$ variants):** Using `evaluation/corruptions.py` (`seed=42`), we generate all 10 realistic perturbation scenarios across 3 monotonic severity levels (Table 2), creating 9,000 corrupted images from all 300 Source A plates. Under `perspective_tilt`, bounding boxes are reprojected via the exact 3×3 homography matrix H and linked via explicit `original_annotation_id` lineage (`0` dropped boxes across all 300×3 transforms). To evaluate the full 64-cell matrix sequentially within a single-process run, we sample a fixed cohort of $N = 100$ Source A plates (`seed=42`, preserving source balance with 54 cars / 46 motorcycles and 47 one-line / 53 two-line plates, shared cohort hash `d62623e9359cba80`), evaluated alongside clean Source A and Source B at Pareto resolution 480×480.
3. **Source B (Real-World Adverse Field Sanity Check, $N = 60$):** Sixty independent field captures stratified equally into four real-world degradation slices (15 images per slice): *Low Light* (mean gray brightness 61.8, range 51.8–67.7; 12/15 two-line), *Motion / Defocus Blur* (mean Laplacian sharpness 219.0; 12/15 two-line), *High Glare / Overexposure* (mean brightness 145.6; 13/15 two-line), and *Geometry / Perspective Tilt* (15/15 two-line motorcycle plates). Overall, 52 of 60 Source B plates (86.7%) are two-line plates.

4.3 Formal Quantitative Metrics

All evaluation quantities are implemented in `evaluation/metrics.py` using unified matching rules:

- **Detection (mAP@50, Precision, Recall):** Detections within the same image and class are greedily matched to ground-truth boxes by descending confidence with IoU ≥ 0.50. Precision and Recall evaluate predictions above $\tau_{\text{ocr}} = 0.25$, while AP@50 integrates the all-points monotonically interpolated precision envelope across all candidate confidences $\geq \tau_{\text{det}} = 0.001$.
- **End-to-End Exact Match (EM) and Character Error Rate (CER):** After NFC normalization, uppercasing, and removal of whitespace, dots, and hyphens, one-to-one box matching pairs predicted strings $\hat{s}_i$ to ground truth s_i. Missed

Table 2 Summary of the 10 synthetic corruption scenarios evaluated across severity Levels 1, 2, and 3.

Category	Scenario ID	Physical Transform Across Levels 1 → 3
Optics & Light	`low_light`	HSV Value scaling + gamma darkening ($\gamma < 0.6$)
	`overexposure`	Localized Gaussian glare blob over plate neighborhood
	`uneven_shadows`	Directional alpha mask reducing local luminance 40%–60%
Blur & Focus	`motion_blur`	Directional linear motion kernel ($k \in \{7, 15, 25\}$)
	`defocus_blur`	Isotropic Gaussian blur ($k \in \{5, 9, 15\}$)
Weather & Channel	`rain_streaks`	Translucent directional rain streaks (60°–80°)
	`sensor_noise`	Zero-mean additive Gaussian noise ($\sigma \in \{15, 30, 50\}$)
	`jpeg_compression`	Low-bitrate JPEG re-encoding ($Q \in \{30, 15, 5\}$)
Geometry & Occl.	`perspective_tilt`	Homography warp ($\approx 15°, 30°, 45°$) + box reprojection
	`partial_occlusion`	Random dark blotches/scratches masking 5%–15% plate area

ground-truth plates receive $\hat{s}_i = \varnothing$ (counting all $|s_i|$ characters as deletions). With Levenshtein edit distance $d(\hat{s}_i, s_i)$ [9]:

$$\mathrm{EM} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}[\hat{s}_i = s_i] \times 100\%, \qquad \mathrm{CER} = \frac{\sum_{i=1}^{N} d(\hat{s}_i, s_i)}{\sum_{i=1}^{N} |s_i|}. \tag{3}$$

Both metrics are computed overall and sliced separately over `one_line` vs. `two_line` plates.

- **Robustness Degradation and Localization Drift:** Relative EM degradation and bounding-box drift ΔIoU are defined over matched plate identities on clean (B_{clean}) and corrupted (B_{corr}) pairs (after projecting B_{clean} through H under `perspective_tilt`):

$$\mathrm{Deg} = \frac{\mathrm{EM}_{\text{clean}} - \mathrm{EM}_{\text{corr}}}{\mathrm{EM}_{\text{clean}}} \times 100\%, \qquad \Delta\mathrm{IoU} = 1 - \mathrm{IoU}(B_{\text{clean}}, B_{\text{corr}})\,. \tag{4}$$

5 Experimental Results and Analysis

5.1 Model Compression and Memory-to-Compute Shift (Phases 2–3)

Table 3 summarizes detector compression gains measured via `verify_comparison.py` and `benchmark_edge_resources.py`. Moving from PyTorch FP32 (`best.pt`) to Pure ONNX INT8 cuts initialization RAM by **83.8%** (330.96 MB → 53.61 MB) while keeping bounding-box IoU consistency at **97.8%** on reference samples. Note that the initial static ONNX graph (11.70 MB → 3.20 MB, −72.6%) and the dynamic-shape ONNX graph used in Phases 4–6 (11.94 MB → 3.44 MB, −71.2%) achieve essentially identical weight compression.

Table 4 reports Docker cgroup stress-testing results prior to OCR de-bloating. Pure ONNX INT8 executes safely without OOM termination all the way down to **256 MB**

Table 3 Model compression, weight footprint, initialization RAM, and bounding-box fidelity (Phase 2).

Detector Variant	Weight Size	Compression	Init RAM	Fidelity vs. FP32
PyTorch Baseline (`best.pt`)	5.97 MB	Baseline	330.96 MB	IoU 100%, Conf 91.7%
Pruned L1 25% (`best_pruned.pt`)	11.78 MB	24.91% sparsity	–	Conf 90.5% (750k zeros)
ONNX FP32 (static / dynamic)	11.70/11.94 MB	Graph fused	∼110.0 MB	IoU 99.1%, Conf 94.3%
ONNX INT8 (static / dynamic)	**3.20/3.44 MB**	−72.6%/−71.2%	**53.61 MB** (−83.8%)	**IoU 97.8%, Conf 92.2%**

Table 4 Docker cgroup memory/CPU stress-test profiling before OCR optimization (Phase 3).

Emulated Tier	Cgroup Quota	Det (INT8)	Default OCR	Total	OOM
Tier 1: RPi 4	4.0C / 2048 MB	281.34 ms	5,060.00 ms	5.34 s	Survived
Tier 2: RPi 3 B+	2.0C / 1024 MB	453.68 ms	9,486.00 ms	9.94 s	Survived
Tier 3: RPi Zero 2W	1.0C / 512 MB	1,095.84 ms	16,574.00 ms	17.67 s	Survived
Tier 4: Legacy 256M	1.0C / 256 MB	1,000.90 ms	33,406.85 ms	34.40 s	**0 OOM**

RAM (a single-core legacy ceiling where PyTorch crashes during load). Crucially, however, while detector execution takes only 281.34 ms (Pi 4) to 1,000.90 ms (extreme single-core), default RapidOCR dominates runtime at 5.06 s to 33.41 s (**85%–95%** of pipeline latency), confirming a sharp transition from memory-bound to compute-bound behavior.

5.2 Resolution Scaling, SIMD Threading, and OCR Speedup (Phases 4–5)

Table 5 isolates detector latency across input resolutions and thread counts. Reducing input resolution from 640×640 to 480×480 accelerates YOLOv8 INT8 by 1.64× (Pi 4), 2.07× (Pi 3), and 1.85× (Pi 2) while actually improving sample confidence from 59.2% to 60.3% and keeping 100% OCR fidelity; conversely, dropping further to 320 × 320 degrades edge sharpness and confidence (41.7%). Thread-scaling results further prove that ONNX Runtime `intra_op_num_threads` must match physical core count: forcing 4 threads onto a 2.5-core (Pi 3) or 1.5-core (Pi 2) quota increases detector latency from 114.28 ms to 174.01 ms and from 151.58 ms to 299.32 ms, respectively, due to thread synchronization contention.

Applying our four-step OCR de-bloating strategy (Section 3.4) slashes OCR latency by 5.85× on Pi 4 (3,322.86 ms → 568.42 ms), 4.89× on Pi 3 (5,081.26 ms → 1,040.14 ms), and 4.71× on Pi 2 (8,240.25 ms → 1,749.40 ms) with 100% OCR parity on benchmark plates (`68G1-668.86`, `51A-222.22`). Combining INT8 480 × 480 detection with tuned RapidOCR drops total end-to-end latency from 4,184.24 ms to **668.74 ms (∼ 1.50 FPS, 6.26× overall speedup)** on Pi 4, from 6,597.99 ms to **1,154.42 ms**

Table 5 Dynamic resolution scaling and intra-op thread pinning across edge hardware tiers (Phase 4).

Emulated Tier	Resolution Scaling (ms / Speedup vs. 640)				Threads @ 480 (ms)		
	640	480 (Pareto)	384	320	1T	2T	4T
RPi 4 (4C/2G)	187.43	**114.17** (1.64×)	68.64 (2.73×)	59.94 (3.13×)	154.54	118.12	**100.32**
RPi 3 (2.5C/1G)	236.63	**114.33** (2.07×)	77.39 (3.06×)	60.06 (3.94×)	155.56	**114.28**	174.01
RPi 2 (1.5C/512M)	296.99	**160.54** (1.85×)	102.22 (2.91×)	70.57 (4.21×)	**151.58**	161.50	299.32

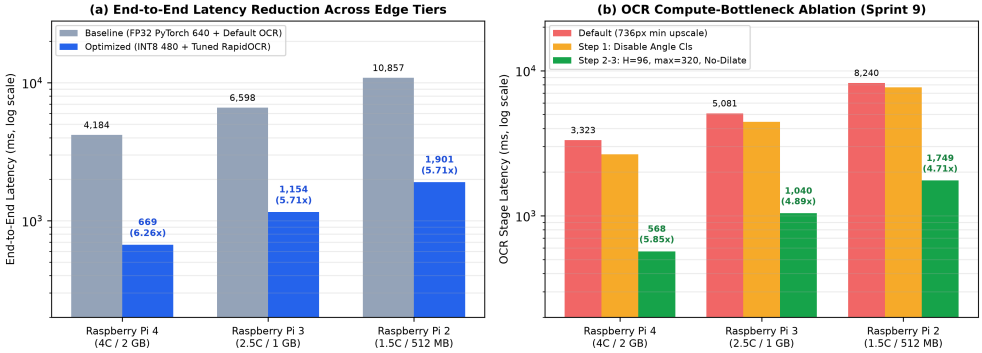


Fig. 2 Phase 5 throughput breakthroughs under Raspberry Pi cgroup profiles: (a) overall end-to-end latency reduction (5.71×–6.26× speedup) and (b) stage-wise OCR ablation proving that eliminating DBNet 736 px minimum-side upscaling yields a 4.71×–5.85× speedup.

(5.71×) on Pi 3, and from 10,856.81 ms to **1,900.98 ms** (5.71×) on Pi 2, as depicted in Fig. 2.

5.3 Full Clean Multi-Resolution Baseline (Source A & Source B)

Table 6 presents the complete 12-run clean multi-resolution benchmark across all 300 Source A and 60 Source B plates (2,160 total inferences on the `x86_64` host, `threads=4`), accompanied by Fig. 3. Three structural findings emerge:

1. **Resolution 480×480 improves precision while preserving EM:** On Source A ($N = 300$), scaling from 640 to 480 reduces detector false positives by over two-thirds, boosting Precision from 85.37%–85.80% to **94.44%–94.74%**, while holding mAP@50 near 96% (96.32% FP32, 95.94% INT8) and overall EM above 70% (72.33% FP32, 70.33% INT8). Further scaling to 320 cuts Recall on small motorcycle plates (lowering moto mAP@50 from 91.9%–92.5% at 640 down to 88.4%–90.1% at 320).
2. **Car (1-line) vs. Motorcycle (2-line) disparity:** Across all resolutions on Source A, cars ($n = 150$) attain **100.00% mAP@50** (640 and 480) and **94.00%–95.33% EM** (`one_line` EM 95.56%–97.04%, CER 0.0037–0.0112). Motorcycles ($n = 150$, `two_line`), by contrast, achieve 88.37%–92.52% mAP@50 yet drop to

Table 6 Full clean multi-resolution benchmark across Source A (SA, $n = 300$) and Source B (SB, $n = 60$) on `x86_64` CPU (4T). 1L/2L denote one-line/two-line EM; latency reports detection, OCR, and end-to-end total means.

Src (N)	Mdl	Res	mAP (@50%)	Prec. (%)	Rec. (%)	EM (%)	1L (%)	2L (%)	CER	Det (ms)	OCR (ms)	Total (ms/FPS)
SA (300)	FP32	640	96.91	85.37	95.33	71.00	97.04	49.70	0.1128	112.76	82.56	207.58/4.82
SA (300)	FP32	480	96.32	94.44	90.67	72.33	95.56	53.33	0.1414	95.07	69.15	172.04/5.81
SA (300)	FP32	320	95.10	93.99	88.67	68.33	96.30	45.45	0.1679	58.59	60.16	122.74/8.15
SA (300)	INT8	640	96.57	85.80	94.67	70.67	96.30	49.70	0.1172	244.84	91.32	347.40/2.88
SA (300)	INT8	480	95.94	94.74	90.00	70.33	96.30	49.09	0.1510	140.27	70.00	217.34/4.60
SA (300)	INT8	320	95.54	97.05	87.67	68.67	96.30	46.06	0.1696	104.52	65.61	174.45/5.73
SB (60)	FP32	640	63.10	73.47	60.00	36.67	62.50	32.69	0.5169	148.13	70.96	232.66/4.30
SB (60)	FP32	480	60.70	75.00	50.00	30.00	37.50	28.85	0.5753	96.17	51.06	157.10/6.37
SB (60)	FP32	320	54.68	84.85	46.67	35.00	50.00	32.69	0.5663	52.24	50.59	107.34/9.32
SB (60)	INT8	640	**65.95**	78.72	**61.67**	**38.33**	62.50	**34.62**	**0.4652**	227.86	61.31	303.98/3.29
SB (60)	INT8	480	62.11	76.92	50.00	28.33	37.50	26.92	0.5910	129.88	53.94	191.82/5.21
SB (60)	INT8	320	56.40	87.10	45.00	31.67	25.00	32.69	0.5843	83.83	45.44	132.37/7.55

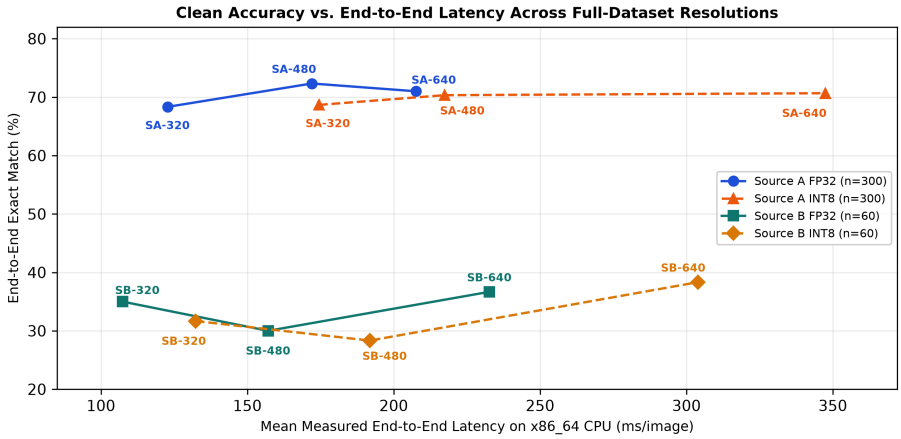


Fig. 3 Clean accuracy vs. measured mean end-to-end latency on `x86_64` CPU across all 12 full-dataset configurations ($N = 300$ Source A and $N = 60$ Source B at 640, 480, and 320).

42.00%–50.67% EM (CER 0.205–0.315), establishing two-line stacked recognition as the primary accuracy bottleneck.

3. **x86 AVX2 vs. ARM quantization trade-off:** On the desktop `x86_64` host, `QUInt8` detection executes slower than FP32 (e.g., 140.27 ms vs. 95.07 ms at 480 on Source A) because standard AVX2 favors FP32 vector instructions when VNNI INT8 kernels are absent, forcing per-op dequantization overhead. Nevertheless, INT8 substantially lowers peak memory (117.21 MiB vs. 335.07 MiB at 480; noting that sequential in-process RSS reuse attenuates later cell baselines) and yields higher mAP@50 on the hard real-world Source B dataset across every resolution (65.95% vs. 63.10% at 640).

Table 7 End-to-end robustness across 10 corruption scenarios and 3 severity levels (480×480, $N = 100$ shared Source A cohort; clean EM = 74.00% for both FP32 (F) and INT8 (I)). Percentage signs omitted in columns marked (%).

		mAP@50 (%)		EM (1L/2L) (%)		CER		Deg (%)		ΔIoU
Scenario	L	FP32	INT8	FP32	INT8	FP32	INT8	FP32	INT8	(F/I)
low_light	1	97.97	97.66	69.0(93.6/47.2)	70.0(91.5/50.9)	0.1322	0.1432	6.76	5.41	.020/.021
	2	95.53	95.03	65.0(87.2/45.3)	67.0(87.2/49.1)	0.1726	0.1628	12.16	9.46	.050/.050
	3	56.58	59.75	0.0(0.0/0.0)	0.0(0.0/0.0)	1.0000	1.0000	100.00	100.00	.126/.124
overexposure	1	97.82	97.82	74.0(97.9/52.8)	73.0(95.7/52.8)	0.1346	0.1346	0.00	1.35	.006/.010
	2	97.97	97.79	63.0(87.2/41.5)	61.0(85.1/39.6)	0.1701	0.1934	14.86	17.57	.032/.031
	3	51.43	50.94	5.0(8.5/1.9)	4.0(6.4/1.9)	0.8825	0.8800	93.24	94.59	.213/.226
uneven_shadows	1	97.88	97.76	71.0(93.6/50.9)	71.0(93.6/50.9)	0.1359	0.1603	4.05	4.05	.016/.019
	2	95.12	95.03	69.0(97.9/43.4)	71.0(95.7/49.1)	0.1652	0.1554	6.76	4.05	.034/.034
	3	91.88	91.36	63.0(93.6/35.9)	63.0(93.6/35.9)	0.2766	0.2876	14.86	14.86	.046/.048
motion_blur	1	95.85	96.00	22.0(34.0/11.3)	23.0(34.0/13.2)	0.3525	0.3525	70.27	68.92	.077/.079
	2	95.74	94.12	2.0(0.0/3.8)	2.0(0.0/3.8)	0.9523	0.9547	97.30	97.30	.127/.127
	3	92.46	93.09	1.0(0.0/1.9)	1.0(0.0/1.9)	0.9792	0.9743	98.65	98.65	.174/.178
defocus_blur	1	97.15	97.23	62.0(91.5/35.9)	67.0(93.6/43.4)	0.1432	0.1530	16.22	9.46	.039/.040
	2	93.06	93.64	34.0(44.7/24.5)	34.0(48.9/20.8)	0.3660	0.3660	54.05	54.05	.101/.103
	3	82.37	82.46	2.0(0.0/3.8)	2.0(0.0/3.8)	0.9339	0.9498	97.30	97.30	.157/.158
rain_streaks	1	98.45	98.36	76.0(95.7/58.5)	73.0(95.7/52.8)	0.1200	0.1212	-2.70	1.35	.008/.013
	2	98.21	98.39	69.0(93.6/47.2)	68.0(93.6/45.3)	0.1506	0.1518	6.76	8.11	.025/.027
	3	66.75	66.29	14.0(25.5/3.8)	10.0(21.3/0.0)	0.7650	0.7882	81.08	86.49	.084/.087
sensor_noise	1	99.61	99.31	72.0(93.6/52.8)	66.0(91.5/43.4)	0.1236	0.1395	2.70	10.81	.031/.031
	2	98.35	97.67	53.0(78.7/30.2)	52.0(78.7/28.3)	0.2203	0.2485	28.38	29.73	.046/.049
	3	97.44	97.23	33.0(53.2/15.1)	37.0(57.5/18.9)	0.3672	0.3758	55.41	50.00	.060/.059
jpeg_compression	1	98.08	98.10	73.0(93.6/54.7)	72.0(91.5/54.7)	0.1297	0.1200	1.35	2.70	.011/.013
	2	99.03	98.78	66.0(85.1/49.1)	68.0(85.1/52.8)	0.1273	0.1310	10.81	8.11	.014/.018
	3	98.22	98.56	48.0(61.7/35.9)	48.0(61.7/35.9)	0.2179	0.2326	35.14	35.14	.032/.034
perspective_tilt	1	96.36	95.65	68.0(95.7/43.4)	69.0(95.7/45.3)	0.1689	0.1775	8.11	6.76	.048/.049
	2	95.35	94.95	68.0(95.7/43.4)	63.0(91.5/37.7)	0.1958	0.1983	8.11	14.86	.069/.069
	3	81.37	80.44	56.0(87.2/28.3)	57.0(89.4/28.3)	0.3280	0.3207	24.32	22.97	.119/.120
partial_occlusion	1	96.95	96.99	34.0(46.8/22.6)	28.0(42.6/15.1)	0.2840	0.2950	54.05	62.16	.016/.020
	2	95.04	95.29	11.0(17.0/5.7)	12.0(19.2/5.7)	0.3709	0.3709	85.14	83.78	.027/.030
	3	94.05	93.63	4.0(4.3/3.8)	1.0(0.0/1.9)	0.5410	0.5667	94.59	98.65	.037/.040

5.4 Ten-Corruption Robustness Matrix: FP32 vs. INT8 Parity

Tables 7 and 8 report the full 30-corruption matrix at Pareto resolution 480×480 over the identical $N = 100$ Source A cohort (where clean baseline yields EM = 74.00% for both models, mAP@50 = 97.84% FP32 vs. 97.48% INT8). Figure 4 plots relative degradation trajectories across severity levels 1–3, and Fig. 5 displays pairwise FP32 vs. INT8 radar envelopes at each severity level.

Three key conclusions emerge from Tables 7–8 and Figs. 4–5:

1. **Quantization preserves robustness:** Over 3,000 corrupted test evaluations per model, dynamic `QUInt8` compression tracks FP32 closely, with a mean signed deviation of only -0.09 **pp** on mAP@50 and -0.47 **pp** on EM (mean absolute EM difference 1.80 pp; maximum cell delta ± 6.00 pp, with INT8 outperforming FP32 on `defocus_blur L1` by $+5.0$ pp and `sensor_noise L3` by $+4.0$ pp).
2. **Stage-wise decoupling between Detector and OCR resilience:** YOLOv8 maintains $>$ 91% mAP@50 even under severe `sensor_noise-3` (97.2%–97.4%), `jpeg_compression-3` (98.2%–98.6%), `partial_occlusion-3` (93.6%–94.1%), and `motion_blur-3` (92.5%–93.1%), breaking down only at extreme

Table 8 Aggregated FP32 vs. INT8 robustness parity across severity levels (10 perturbations per level, 3,000 corrupted evaluations per model).

Scope	Mdl	Mean mAP (%)	Mean EM (%)	EM 1L / 2L (%)	Mean CER	Mean Deg (%)	Δ(INT8−FP32) (mAP / EM)
Level 1	FP32	97.61	62.10	83.62 / 43.02	0.1725	16.08	−0.12 pp / −0.90 pp
	INT8	97.49	61.20	82.55 / 42.26	0.1797	17.30	
Level 2	FP32	96.34	50.00	68.72 / 33.40	0.2891	32.43	−0.27 pp / −0.20 pp
	INT8	96.07	49.80	68.51 / 33.21	0.2933	32.70	
Level 3	FP32	81.26	22.60	33.40 / 13.02	0.6291	69.46	+0.12 pp / −0.30 pp
	INT8	81.38	22.30	32.98 / 12.83	0.6376	69.86	
All 30	**FP32**	**91.74**	**44.90**	**61.91 / 29.81**	**0.3636**	**39.32**	**−0.09 pp / −0.47 pp**
	INT8	**91.64**	**44.43**	**61.35 / 29.43**	**0.3702**	**39.95**	

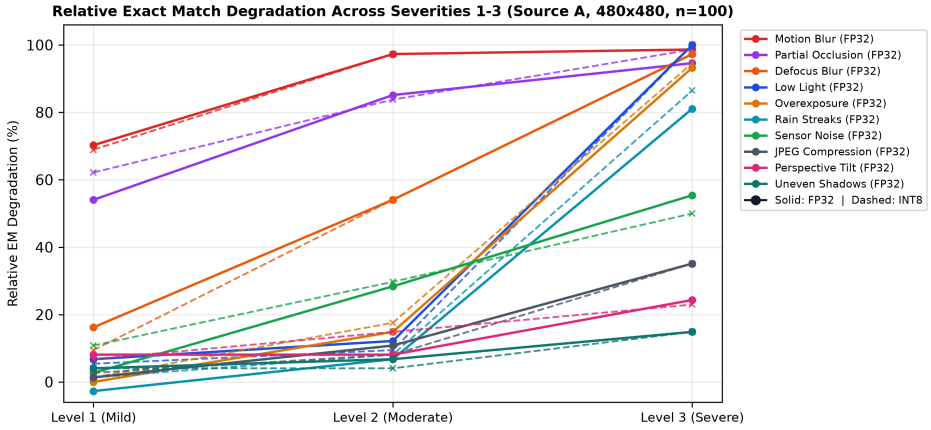


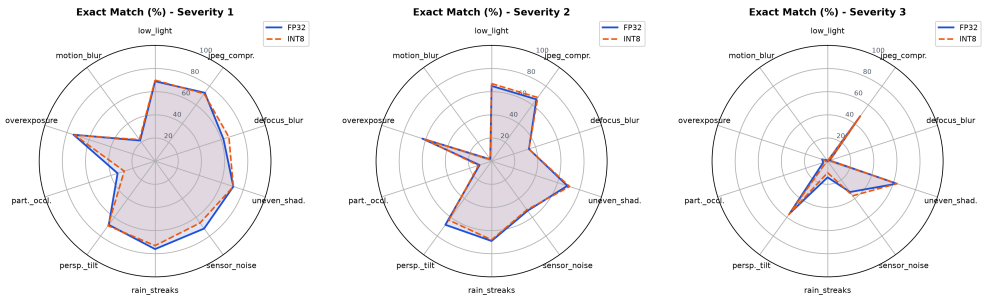
Fig. 4 Relative Exact Match degradation (%) across corruption severities 1–3 on the shared Source A cohort (480 × 480).

Level 3 optical losses (`overexposure-3`: 50.9%–51.4%, `low_light-3`: 56.6%–59.8%, `rain_streaks-3`: 66.3%–66.8%). Conversely, RapidOCR collapses far earlier under `motion_blur` (EM drops from 74.0% to 22.0%–23.0% already at Level 1 despite ∼ 96% mAP@50) and `partial_occlusion` (28.0%–34.0% at Level 1). Meanwhile, `uneven_shadows-3` (63.0% EM) and `perspective_tilt-3` (56.0%–57.0% EM) remain comparatively tolerable.

3. **Persistent 1-line vs. 2-line gap:** Across all 30 corrupted environments, one-line plates average 61.91% EM (FP32) / 61.35% EM (INT8)—more than double the accuracy on two-line plates (29.81% / 29.43%).

5.5 Real-World Sanity Check on Source B

Table 9 and Fig. 6 dissect performance across the 4 real-world challenges of Source B ($N = 60$). Both models behave consistently across resolutions, with INT8 matching or slightly outperforming FP32 in detection across all four categories at 640 and 480.



(a) Level 1 (Mild) (b) Level 2 (Moderate) (c) Level 3 (Severe)

Fig. 5 End-to-end Exact Match (%) radar profiles comparing FP32 and INT8 across all 10 environmental perturbations at severity Levels 1, 2, and 3 (480×480, $N = 100$).

Table 9 Source B ($n = 60$) slice evaluation across four physical field conditions (15 images/slice; metric pairs denote FP32/INT8 in %).

			640 (F/I %)		480 (F/I %)		320 (F/I %)	
Slice ($n = 15$)	**2L**	**Bri./Shp.**	**mAP**	**EM**	**mAP**	**EM**	**mAP**	**EM**
Low Light	12/15	61.8/492.9	80.31/80.31	60.00/53.33	76.81/76.83	46.67/46.67	70.62/70.62	53.33/40.00
Motion / Defocus Blur	12/15	93.8/219.0	65.91/65.52	46.67/46.67	71.90/71.90	40.00/40.00	67.95/67.94	46.67/40.00
High Glare / Overexp.	13/15	145.6/399.9	48.55/**54.00**	33.33/**40.00**	32.01/35.43	26.67/20.00	42.34/43.70	33.33/33.33
Geometry / Persp. Tilt	15/15	102.0/780.8	60.05/**66.41**	6.67/**13.33**	65.30/66.37	6.67/6.67	42.01/46.81	6.67/13.33
Overall (60)	**52/60**	–	63.10/**65.95**	36.67/**38.33**	60.70/62.11	30.00/28.33	54.68/56.40	35.00/31.67

Critically, real-world EM (28.3%–38.3%) falls well below Source A (70.3%–72.3%) due to two compounded factors: (i) 86.7% of Source B comprises stacked two-line plates, and (ii) the *Geometry / Perspective Tilt* group consists 100% (15/15) of steeply angled two-line motorcycle plates in cluttered backgrounds, yielding 6.67%–13.33% EM even when detector mAP@50 holds at 60.1%–66.4%. Similarly, physical headlight glare (*High Glare*) induces blooming saturation that degrades 480×480 mAP@50 to 32.0%–35.4% (intermediate between synthetic `overexposure` Levels 2 and 3). This contrast refutes overly optimistic weather accuracy claims (80.9%–89.0%) reported without strict End-to-End Exact Match or two-line separation in Paper 23 [1].

5.6 Failure Mode Diagnosis and Threats to Validity

Inspecting 3,840 failure records exported by `evaluation/error_analysis.py` across all 64 robustness cells reveals three distinct failure mechanisms:

1. **Optical Detector Collapse:** At `low_light-3` and `overexposure-3`, edge contrast vanishes, generating 68 and 65 missed detections (INT8-480), respectively. Even the 32 plates detected under `low_light-3` yield empty strings (e.g., GT `65B1-045.66` predicted as `""` at confidence 0.938) because cropped ROI luminance falls below DBNet's binarization threshold.

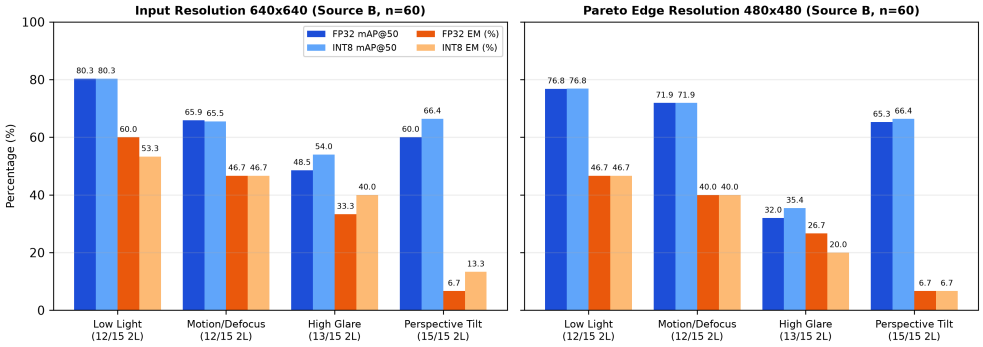


Fig. 6 Source B real-world slice comparison across four physical gate challenges at 640×640 and Pareto 480×480, demonstrating FP32/INT8 parity alongside severe two-line geometry and glare difficulty.

2. **Early OCR Collapse Despite High-Confidence Detection:** Under `motion_blur-1` (only 8 misses, 69 OCR mismatches for INT8) and `partial_occlusion-1` (9 misses, 63 OCR mismatches), YOLOv8 retains $\sim$ 0.94 confidence while character strokes smear or drop (e.g., `51G-222.37` $\rightarrow$ `51G-272.37`, `51G-436.30` $\rightarrow$ `51G-4360`). On clean two-line plates, DBNet frequently omits or merges upper-row series tokens (e.g., `94R1-9764` $\rightarrow$ `9764`, `63C1-3900` $\rightarrow$ `63C`).
3. **Axis-Aligned Box Drift Under Extreme Perspective:** Under `perspective_tilt-3` ($\approx 45°$), axis-aligned ground-truth bounding boxes expand asymmetrically around slanted plates. Tighter detector predictions drop marginally below IoU = 0.50, simultaneously generating 49 high-confidence "false positives" (up to confidence 0.945) and 26 missed detections even when ΔIoU on matched pairs stays 0.120.

Threats to Validity: First, Source A OCR text labels carry `labels_verified=false` (`quality_status="exploratory"`) because verified COCO geometry was paired with curated OCR annotations; absolute Source A EM ceilings should therefore be interpreted relative to clean baseline deltas rather than human-audited ground truth. Second, while RPi hardware optimizations were measured under cgroup-limited Linux containers, the 64-cell robustness run executed on an `x86_64` host CPU, where INT8 speedups do not manifest due to missing AVX-512 VNNI kernels. Third, sequential cell execution reuses Python allocator/OS caches, so observed RSS numbers denote process-level runtime observations rather than isolated per-cell cold boots.

6 Conclusion and Future Work

This paper resolved critical efficiency, reproducibility, and evaluation blind spots in edge Vietnamese ALPR deployment. Replacing PyTorch with dynamic INT8 ONNX Runtime reduces initialization RAM by 83.8% and weight files by over 71% while surviving 256 MB memory constraints. More importantly, diagnosing hidden OCR

over-processing—eliminating RapidOCR's default 736 px minimum-side upscaling via $H_{\max} = 96$ px ROI capping, `limit=320 max`, disabled dilation, and skipped angle classification alongside 480×480 Pareto detector scaling and core-matched SIMD threads—accelerates OCR by 5.85× and overall Raspberry Pi 4 pipeline speed by 6.26× to 668.74 ms. Finally, our transparent two-tier robustness evaluation across 9,000 synthetic corrupted variants and 60 adverse real-world captures confirms that INT8 quantization preserves robustness identically to FP32 (ΔmAP@50 $= -0.09$ pp, ΔEM $= -0.47$ pp), while identifying OCR sensitivity to motion blur, partial occlusion, and skewed two-line motorcycle plates as top priorities for future research.

Future improvements will pursue: (i) lightweight ROI perspective rectification and adaptive local CLAHE preprocessing triggered selectively on low-luminance ROIs prior to DBNet; (ii) specialized CTC/attention recognizers trained natively on rectified two-line Vietnamese plates; (iii) full manual human audit of Source A OCR ground truth; and (iv) on-device ARM64 verification directly on physical Raspberry Pi 4/5 hardware.

Acknowledgments. The authors thank all members of the Embedded Computer Vision research team for collaborative engineering across dataset curation, synthetic perturbation generation, metric verification, benchmark orchestration, visualization, and Docker container packaging.

Declarations

- **Funding:** This research received no external funding.
- **Conflict of interest:** The authors declare no competing financial or non-financial interests.
- **Data and code availability:** Benchmark scripts, corruption generators, evaluation metrics, unit tests, and dataset manifests are maintained in the project repository (https://github.com/HuyHo1405/260907_MSS301_Research).

References

[1] Jagtap, P.B., Dhawale, S.D.: YOLO-based approach for helmet and number plate detection using Raspberry Pi. Computer Research and Development **25**(10), 88–106 (2025)

[2] Smith, R.: An overview of the Tesseract OCR Engine. In: Ninth International Conference on Document Analysis and Recognition (ICDAR 2007), vol. 2, pp. 629–633. IEEE Computer Society, Washington, DC, USA (2007). https://doi.org/10.1109/ICDAR.2007.4376991

[3] Liao, M., Wan, Z., Yao, C., Chen, K., Bai, X.: Real-time scene text detection with differentiable binarization. In: Proceedings of the AAAI Conference on Artificial Intelligence (AAAI-20), vol. 34, pp. 11474–11481 (2020). https://doi.org/10.1609/aaai.v34i07.6812

[4] Li, C., Liu, W., Guo, R., Yin, X., Jiang, K., Du, Y., Du, Y., Zhu, L., Lai, B., Hu, X., Yu, D., Ma, Y.: PP-OCRv3: More Attempts for the Improvement of Ultra Lightweight OCR System (2022)

[5] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics. Version 8.0.0 (2023)

[6] Zherzdev, S., Gruzdev, A.: LPRNet: License Plate Recognition via Deep Neural Networks (2018)

[7] Hendrycks, D., Dietterich, T.G.: Benchmarking neural network robustness to common corruptions and perturbations. In: International Conference on Learning Representations (ICLR) (2019)

[8] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft COCO: Common objects in context. In: Computer Vision – ECCV 2014. Lecture Notes in Computer Science, vol. 8693, pp. 740–755. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-10602-1_48

[9] Levenshtein, V.I.: Binary codes capable of correcting deletions, insertions, and reversals. Soviet Physics – Doklady **10**(8), 707–710 (1966)